

Chapitre I: Introduction au langage PL SQL

1. Introduction

SQL est un langage complet pour travailler sur une base de données, mais il ne comporte pas des instructions procédurales (telles que les appels de procédures, les instructions de contrôles etc..) : c'est langage ensembliste de quatrième génération.

Oracle dispose d'un nouveau langage appelé PL/SQL (Procedural Langage SQL) qui est une extension procédurale du langage SQL. Cette extension est composée des commandes de LMD de SQL , d'opérateurs, des curseurs et de traitement de transactions, auxquelles sont ajoutées des instructions d'affectation , conditionnelles , répétitives et de branchement. De ce fait PL/SQL intègre les instructions des langages de programmation traditionnels pour permettre des traitements procéduraux et pour faire de lui un langage de traitement de transactions permettant de grouper un ensemble de commandes (dans une structure de bloc) et de les soumettre au noyau . Ce bloc sera considéré comme une transaction dont l'exécution est indivisible : elle s'achève totalement et jamais partiellement.

Le langage PL/SQL est donc un langage algorithmique complet qui comprend :

- la partie LID de SQL (Select) et ses extensions
- la partie LMD de SQL (Update, Insert, Delete)
- Fonctions standards du SQL
- La gestion de transactions (commit, rollback, savepoint)
- Une partie procédurale : les instructions PL permettant de gérer :
 - des boucles (LOOP, FOR, WHILE, EXIT)
 - des conditionnelles (IF, THEN, ELSEIF, ELSE, ENDIF)
 - des sous programmes (Blocs PL/SQL)
 - des calculs,
 - des fonctions et des packages(modules)

Notez en fin que le PL/SQL ne comporte pas d'instructions du LDD (Create, Alter, Rename) ni les instructions de contrôles (Grant et Revoke) ce qui veut que dans un programme PL/SQL (on l'appelle un bloc PL/SQL) on ne trouve jamais des telles commandes.

2. Structure d'un Bloc PL/SQL

PL/SQL n'interprète pas une commande, mais un ensemble de commandes contenues dans un programme appelé bloc PL/SQL.

La structure d'un bloc est la suivante :

```
Declare
    Déclaration de variables, constantes, exception;
Begin
    Section obligatoire contenant des commandes exécutables
    Instructions SQL ;
    Instructions PL/SQL;
    Possibilité de bloc fils ( imbrication de bloc );
Exception
    Traitement des exceptions ( Gestion des erreurs )
End;
/
```

Remarques

- Les sections Declare et Exception sont optionnelles (Au minimum : BEGIN et END ;)
- Chaque instruction de n'importe quelle section (Declare, Begin et Exception) doit se terminer par un point virgule.
- Un problème complexe peut se réduire en plusieurs blocs structurés (simplification des traitements complexes).
- On peut déclarer des blocs de code dans SQL*PLUS

```
begin
dbms_output.put_line('Bonjour');
end;
/
Procédure PL/SQL terminée avec succès.
```

- **dbms.output_line** permet d'afficher du texte.

```
begin
    dbms_output.put_line('Bonjour. Nous sommes le
        ||to_char(sysdate, 'dd month yyyy'));
end;
/
Bonjour. Nous sommes le 09 février 2018
Procédure PL/SQL terminée avec succès.
```

- **Commande « / » :**

Le « / » enregistre le bloc et demande l'exécution du bloc en cours.

Si on rappelle le / , ça exécute le bloc en cours.

Le « / » rappelle toujours la dernière commande passée.

- **Serveroutput à ON**

L'exécution est correcte, mais rien ne s'affiche !

Il faut passer le paramètre « serveroutput » à ON.

```
SQL> show serveroutput
serveroutput OFF
SQL> set serveroutput ON
```

On peut relancer l'exécution avec « / »

```
SQL> /
Bonjour
Procédure PL/SQL terminée avec succès.
SQL>
```

- **Run**

Le RUN affiche le bloc en plus de l'exécuter.

```
SQL> run
1 begin
2 dbms_output.put_line('Bonjour');
3 end;
4 /
Bonjour
Procédure PL/SQL terminée avec succès.
SQL>
```

- **Script**

Les programmes peuvent être saisis directement sous SQLPLUS ou enregistrés dans des scripts.

```
-- test.sql
-- script qui affiche bonjour
begin
dbms_output.put_line('Bonjour');
end;
/
```

Exécution du script avec la commande start ou @

```
SQL> @test
Bonjour
Procédure PL/SQL terminée avec succès.
SQL>
```

- **Commentaires**

```
/* script de définition d'une procédure  
procédure « bonjour » : affiche bonjour,  
usage des commentaires en style C  
*/  
-- commentaires derrière deux tirets et un espace
```

- **Instructions SQL**

- Pas de SELECT classique en PL-SQL
- Pas de LDD en PL-SQL : CREATE TABLE, DROP TABLE, ALTER TABLE

```
begin  
    select * from emp;  
end;  
/  
select * from emp;  
*  
ERREUR à la ligne 2 :  
ORA-06550: Ligne 2, colonne 1 :  
PLS-00428: une clause INTO est attendue dans cette instruction  
SELECT
```

L'erreur dit : « une clause INTO est attendue » : le INTO ça permettra de mettre le résultat d'un SELECT dans une variable.

- Les instructions du LMD autorisées sont : **INSERT, UPDATE, DELETE**

```
begin  
update emp set comm = comm + 0.1*comm  
where job='directeur commercial';  
commit ;  
end;  
/  
Procédure PL/SQL terminée avec succès.  
SQL>
```

- On vérifie le bon fonctionnement du bloc en regardant les tuples avant et après la Modification.

```
Select * from emp where 'directeur commercial';  
begin  
update emp set comm = comm + 0.1*comm  
where job='directeur commercial';  
commit ;  
end;  
/  
Select * from emp where job='directeur commercial';
```

2. Les Déclarations PL/SQL

La partie déclarative dans un bloc PL/SQL, peut comporter quatre types de déclaration. Elle est délimitée par les mots-clé : 'Declare' qui spécifie son début et 'Begin' qui signifie sa fin et le début des commandes :

Declare

.....

..... Les déclarations

Begin

Les types de déclarations possibles dans cette partie sont les suivantes :

- déclaration des variables et des constantes
- déclaration des tableaux et enregistrements
- déclaration de curseurs,
- déclaration des exceptions.

2.1 Les Identificateurs PL/SQL

Un identificateur permet de nommer un objet PL/SQL tels qu'une constante, une variable, une fonction ou un curseur etc..

un identificateur est une chaîne alphanumérique (lettres, chiffres et caractères spéciaux) commençant obligatoirement par une lettre alphabétique.

Quelques caractères ne sont pas permis dans un identificateur comme : l'espace, \$, / et -. De plus un identificateur doit être différent des mots réservés appartenant au vocabulaire du langage tels que : if, then, else, elsif, min, max, count, etc..

2.2 Types de données

Chaque variable ou constante dans un bloc PL/SQL, possède un type qui peut être scalaire ou composé.

Les types scalaires sont :

a) Les types numériques

Ils représentent des quantités sur lesquelles peuvent s'effectuer des calculs, nous distinguons :

- **number[(n[,m])]** : type numérique général utilisé pour désigner n'importe quel numérique. On peut spécifier la précision (nombre total des chiffres) et l'échelle (nombre des chiffres décimaux). Un numérique de ce type peut atteindre au maximum 38 chiffres .

Dans une déclaration de type 'Number', il est conseillé de spécifier la taille c'est à dire utiliser la déclaration 'Number (n,m)' au lieu de 'Number', sinon la variable lui sera réservée une zone mémoire permettant de stocker le plus grand nombre avec 38 chiffres.

- **décimal, float, integer, real, smallint** sont des sous types de 'Number'

- **binary_integer** : désigne un entier signé compris entre -2147483647 et +2147483647

- **natural et positive** sous types de binary_integer représentant respectivement les entiers naturels et positifs.

b) Les types Caractère

- **Char(n)** : une variable de ce type représente une chaîne de n caractères, pouvant aller jusqu'à 32767 caractères(mais la taille maximale d'une colonne d'une table déclarée de ce type est de 255 caractère au maximum). 'Char' veut dire un caractère .
- **Varchar2(n)** : une variable déclarée de ce type représente une chaîne de caractères de longueur variable allant jusqu'à 32767 caractères. Ce même type de données est limité à 4000 caractères dans la définition de la colonne d'une table
- **Long** est équivalent à varchar2 à l'exception de la taille d'une variable de ce type peut atteindre les 2 GO pour stocker tout un texte.

c) Type Date

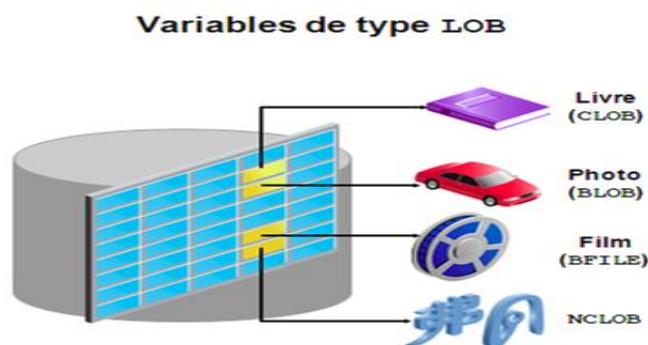
- Date : représentant le type date

d) Type logique

- Boolean : est un type qui stocke true, false ou NULL. Seules les variables peuvent avoir ce type.

e) Type LOB

Les objets LOB (Large Object) sont conçus pour le stockage de grandes quantités de données. Une colonne de base de données peut être de la catégorie LOB. Avec les types de la catégorie LOB (BLOB, CLOB, etc.), vous pouvez stocker des blocs de données non structurées (par exemple du texte, des images, des clips vidéo ou des sons) d'une taille pouvant aller jusqu'à 4 gigaoctets.



Les types composés sont :

- le record : une variable de ce type comporte plusieurs champs de types différents
- le tableau : interprété comme une table .

Ces deux types seront détaillés par la suite.

2.3 Les variables et les constantes**a) Définition**

La déclaration d'une variable consiste à allouer un espace pour stocker et modifier une valeur. Elle est typée et peut recevoir une valeur par défaut et/ou un statut not null. Une constante est définie comme une variable, mais l'utilisateur ne peut pas modifier son contenu.

La syntaxe de déclaration d'une variable (ou une constante) est :

```
Nom_variable[constant] { type |identifiant%type |identifiant%rowtype }
[not null ]
[ { := |default } expression PL/SQL ]
```

- l'attribut **constant** permet de spécifier qu'il s'agit d'une constante et non une variable
- l'attribut **not null** rend obligatoire d'attribuer une valeur à la variable lors de sa définition.
- Une variable peut être définie par référence au type d'une colonne d'une table, ceci est très utile dans les traitements : parfois on voulait affecter une colonne dans une variable et même si on ne connaît pas le type explicite de la colonne il suffit de définir la variable de même type de la colonne sans l'expliciter, ceci grâce à l'instruction : **nom_v table.colonne%type.**
- On peut faire aussi référence à une table par l'instruction : **nom_variable table%rowtype**, ceci sous entend la déclaration d'une variable composée des mêmes champs que la table mentionnée. La variable ainsi déclarée est un record.
- On peut faire référence à une variable précédemment définie par l'instruction : **nom_variable_nouvelle nom_variable_précédente %type.**
- L'initialisation d'une variable se fait par l'opérateur ':= ' suivi de :
 - d'une constante,
 - d'une expression PL/SQL

Exemples de déclaration de variables :

```
Declare
Totalc number(8,3);
Nomc varchar2(15) := 'ben moussa';
Longueur number not null := length(nomc) *2 ;
Datec date;
v_deptno EMP.DEPTNO%TYPE;
xVclt emp%rowtype;
Pi constant number :=3.14;
```

b) Initialisation des variables

Deux possibilités pour renseigner une variable sont disponibles :

- par l'opérateur d'affectation noté ' := ' ; exemple totalc:=12.250;
- par la clause select ... into variable1 [,variable2 ..] from ...

Il faut utiliser le deuxième type d'affectation (select ..into) avec précaution, en effet si le Select retourne plus qu'une ligne alors l'affectation ne s'effectue pas et le système retourne une erreur (TOO_MANY_ROWS) de même si aucune ligne n'est retournée (NO_DATA_FOUND); par contre si le Select retourne une et une seule ligne l'affectation s'effectue correctement.

```
Declare
    f float;
    X number default 123.456;
    maConstante CONSTANT VARCHAR2(20):='constante' ;
    v_deptno EMP.DEPTNO%TYPE;

BEGIN
    dbms_output.put_line('Ma constante : '|| maConstante);
    dbms_output.put_line('Number saisi : '|| x);
    SELECT deptno INTO v_deptno
    FROM dept WHERE deptno=10 ;
    dbms_output.put_line('v_deptno sorti du select : '|| v_deptno);
END;
/
```

2.4 Les types composés (tableau et record)

a) Les tableaux :

Le type tableau en PL/SQL est défini comme une table à une seule colonne de taille illimitée de telle façon que le nombre d'éléments d'un tableau va croître dynamiquement. Les éléments de la table (lignes) sont d'un seul type celui de la colonne. La colonne de la table peut prendre l'un des types précédemment définis (scalaire). Les tableaux peuvent posséder une clé primaire (index) non nommée obligatoirement de type Binary_Integer les tableaux PL/SQL doivent être déclarés en deux étapes :

- déclaration du type de la table
- déclaration d'une table de ce type.

La syntaxe est la suivante :

```
Type <nom_type_tableau> is Table of <type> [not null] index by binary_integer;
Nom_tableau nom_type_tableau;
```

Exemple

```
/* défintion du type tableau */
type nom_article is table of varchar2(15) index by binary_integer;
tab_nom nom_article;
```

Cet exemple définit le type tableau nommé 'nom_article', et définit un tableau nommé tab_nom de ce type .

L'initialisation d'un tableau se fait :

```
Tab_nom(1) := 'crayon';
Tab_nom(2) := 'stylo noir'; etc ..
```

Notons enfin que le type table ne permet pas de définir plus d'une colonne (pas des tableaux multi-dimensionnels), ce qui limite son utilisation.

```
DECLARE
    TYPE type_tabNumber IS table of NUMBER index by binary_integer ;
    v_tab type_tabNumber;
    i number ;
    n number;
BEGIN
    v_tab(1):=0 ;
    FOR i IN 2 .. 5 LOOP
        v_tab(i):= v_tab(i-1)+2;
    END LOOP;
    FOR i IN 1 .. 5 LOOP
        dbms_output.put_line('v_tab["||I||"]="||v_tab(i)||');
    END LOOP;
END;
/
```

b) Le type record (enregistrement)

Par analogie aux tableaux PL/SQL, la déclaration d'un enregistrement se fait également en deux étapes :

- déclaration du type de l'enregistrement
- déclaration de la variable sur le type défini.

La syntaxe de déclaration est la suivante :

```
Type <nom_type_record> is record
(champ <type> [not null], champ <type> .. )
<nom_variable_enreg> nom_type_record;
```

les champs peuvent être de type scalaire (défini précédemment) .

Exemple :

```
Type client_rec is record
( code number, nom client.nom%type, date_nais date, ca client.ca%type ) ;
cl client_rec;
cl.code := 12 ;
cl.nom := 'ben talab';
cl.date_nais:='12-jan-78';
cl.ca:=235.568;
```

Les déclarations (des tableaux et des records se font toujours dans la partie Declare d'un bloc PL/SQL avec les autres variables scalaires).

Remarques:

- Le type enregistrement peut utiliser dans sa définition un autre type enregistrement.

```
Type adresse is record
(
    num_rue number,
    nom_rue varchar2(15),
    ville varchar2(15);
Type client is record
(
    numclt smallint,
    nomclt varchar2(15),
    adrcclt adresse,
..)
```

- Les variables de type RECORD peuvent correspondre à un tuple. On fait un SELECT .. INTO dans le RECORD

```
-- empno à saisir : 7839 par exemple
DECLARE
    TYPE employe IS RECORD
    (
        Empno EMP.EMPNO%TYPE,
        ename EMP.ENAME%TYPE,
        sal EMP.SAL%TYPE
    );
    v_emp type_employe;
BEGIN
    SELECT empno, ename, sal INTO v_emp
    FROM emp
    WHERE empno=&empno;
    dbms_output.put_line(v_emp.ename||', employe n°' || v_emp.empno);
    dbms_output.put_line ('salaire mensuel :' || v_emp.sal) ;
    dbms_output.put_line ('salaire annuel :' || 12*v_emp.sal) ;
END ;
/
```

- %ROWTYPE : déclarer une variable dont le type correspond à la définition d'une table.

On reprend l'exemple précédent, mais avec un ROWTYPE.

```
-- empno à saisir : 7839 par exemple
DECLARE
    v_emp EMP%ROWTYPE;
BEGIN
    SELECT * INTO v_emp
    FROM emp
    WHERE empno=&empno;
    dbms_output.put_line(v_emp.ename||', employe n°' || v_emp.empno);
    dbms_output.put_line ('salaire mensuel :' || v_emp.sal) ;
    dbms_output.put_line ('salaire annuel :' || 12*v_emp.sal) ;
END ;
/
```

Le ROWTYPE oblige à faire un SELECT * INTO : le * est nécessaire pour qu'on ait le même nombre d'attributs.

3. Les variables globales

3.1 Variable globales - DEF - Variables de substitution

- **Principes : &&**

Dans SQL*PLUS, Il existe des variables de niveau session.

On peut les créer avec la commande DEF.

On peut les utiliser dans un SELECT avec un &&.

```
SELECT * FROM emp WHERE EMPNO=&&EMPNO;
```

L'interface SQL*PLUS demande :

```
Enter value for EMPNO: 7839
```

On saisit une valeur et on a le résultat pour la valeur saisie :

```
EMPNO ENAME JOB MGR HIREDATE SAL COMM DEPTNO
-----
7839 KING PRESIDENT 17/11/06 5000 10
```

La variable EMPNO est une variable globale.

Une variable définie avec un && garde sa valeur en dehors de la requête de définition.

```
SQL> DEF EMPNO
DEFINE EMPNO = "7839" (CHAR)
```

C'est un (CHAR).

On a saisi 7839, il est interprété comme une chaîne de caractères.

Si on saisit '7839', ça marche aussi

On peut aussi faire un SELECT FROM DUAL avec &EMPNO

```
SQL> SELECT &EMPNO FROM DUAL;
```

La variable est accessible avec un & (un seul). Si on met && ça marche aussi.

- **Création d'une variable**

```
DEF[INE] maVar = texte
```

La variable est accessible pendant toute la session.

C'est un CHAR (chaîne de caractères). Toutefois, il pourra s'exploiter comme un NUM.

```
DEF[INE] maVar = 30
```

Ou

```
DEF[INE] maVar = DALLAS
```

A noter qu'on peut écrire indifféremment sans guillemets, avec guillemets ou avec apostrophes.

- **Suppression d'une variable**

```
UNDEF[INE] maVar
```

- **Afficher la valeur d'une variable**

```
DEF[INE] maVar
```

- **Afficher toutes les variables**

```
DEF[INE]
```

- **Saisie d'une valeur**

```
ACCEPT maVar1 prompt 'entrez un nombre : '
```

```
ACCEPT maVar2 prompt 'entrez du texte : '
```

PROMPT permet de définir le texte pour la saisie. ACCEPT définit la variable si ça n'a pas déjà été fait.

3.2 Variable globale – VAR

On peut définir des variables qui pourront circuler dans les blocs indépendamment de SQL*PLUS. Toutefois, on peut aussi utiliser ces variables et les déclarer dans SQL*PLUS. On peut les créer avec la commande VAR dans SQL*PLUS.

- **Création de la variable dans SQL*PLUS : VAR [IABLE] [nom [type]]**

```
VAR maVar VARCHAR2(10)
```

- **Utilisation dans un bloc : « :maVar »**

```
Begin
:maVar := 'bonjour';
dbms_output.put_line(:maVar);
end;
/
```

On accède à la variable avec un « : » devant :maVar.

- **Création et initialisation de la variable dans SQL*PLUS**

```
VAR maVar2 VARCHAR2(10);
EXEC :maVar2 := 'bonjour2';
Print :maVar2 ;
```

EXEC permet d'exécuter une commande PLSQL, ici une affectation (:=)

Print permet d'afficher le contenu de la variable dans SQL*PLUS

- **Utilisation dans un bloc :**

```
BEGIN
dbms_output.put_line(:maVar2);
END;
/
```

- **Afficher toutes les variables**

```
VAR[IABLE]
```

4. SQL et PL/SQL

4.1 SELECT INTO : récupération d'une valeur ou d'un tuple de la BD

Dans un programme PL/SQL, l'instruction 'Select' emploie une autre clause qui est 'Select ... Into ..' pour identifier les variables PL qui vont recevoir les données ramenées par l'ordre 'select'. La syntaxe de la structure est :

```
Select expression into <liste_variables> from <liste_tables> where ....
```

La structure 'Select .. Into ...' se comporte comme la structure SQL traditionnelle 'select ' sauf que les données ramenées par le 'Select' doivent être conformes aux variables dans le type et dans le nombre : si on a deux variables, la structure 'Select .. into ..' doit ramener deux données de même type que les deux variables.

Noter enfin que la structure 'Select' dans un bloc 'PL/SQL' est toujours accompagnée par la clause 'Into' est servira pour affecter des valeurs aux variables utilisées.

```
DECLARE
    v_nbEmp integer;
    v_emp emp%rowtype;
BEGIN
    SELECT count(*) into v_nbEmp from emp;
    dbms_output.put_line('nb_emp : '||v_nbEmp);
    SELECT * into v_emp from emp where empno=&numEmp;
    dbms_output.put_line('Employé n° '||v_emp.empno);
    dbms_output.put_line('Nom : '||v_emp.ename||' - salaire : '||v_emp.sal);
END;
/
```

- Une requête qui ne renvoie aucun tuple génère une erreur.

Soit le programme PL/SQL suivant qui affiche le libellé et le prix unitaire augmenté de 20% d'un article donné.

```
Create table T_Res(code varchar2(20), designation varchar2(30), prix number) ;
Accept vccode prompt 'Donner le Code Article : '
Declare
    Vlib article.desart%type ;
    Vpu article.pu%type ;
Begin
    Select desart, pu into vlib,vpu from article where idart=&vcode;
    Vpu:=vpu*1.2;
    Insert into T_Res values (&vcode, vlib, vpu);
    Commit ;
End ;
/
Select*from T_Res ;
```

• Noter bien que la structure 'Select .. into ..' dans ce programme ramène exactement deux données à savoir la désignation et prix unitaire de l'article dans le code est saisie au départ. Les deux données seront récupérées dans les deux variables 'vlib' et 'vpu' qui sont d'ailleurs de même type que les données.

• La variable 'vcode' est globale puisqu'elle est utilisée à l'extérieur du bloc PL/SQL pour saisir le code article. Pour l'affichage on a fait recours à une table

temporaire pour récupérer les résultats et les afficher à l'extérieur du bloc comme c'est le cas de ce programme : c'est la table T_Res).

4.2 Insert into

C'est un INSERT INTO classique.

On peut faire : « VALUES(v_emp) » avec v_emp de type %ROWTYPE.

4.3 Update

C'est un UPDATE classique.

On peut faire « SET ROW = v_emp » avec v_emp de type %ROWTYPE.

4.4 Delete

C'est un DELETE classique.

4.5 Retour du LMD

- **SQL%FOUND**

Vrai (true) si un tuple a été créé ou modifié ou supprimé.

- **SQL%NOTFOUND**

Vrai (true) si aucun tuple n'a été créé ou modifié ou supprimé.

C'est le contraire de SQL%FOUND.

- **SQL%ROWCOUNT**

Renvoie le nombre de tuples qui ont été créés ou modifiés ou supprimés.

5. Les Structures de Contrôles

PL/SQL offre deux types d'instructions de contrôles : les traitements conditionnels et les traitements répétitifs .

5.1 Les Traitements Conditionnels

Les instructions conditionnelles doivent permettre de contrôler le moment où une séquence des commandes est exécutée si une condition est vraie, sinon c'est éventuellement une autre séquence qui sera exécutée.

La syntaxe est :

```
If condition then
- Traitement A ;
Else
- Traitement B;
End if ;
```

Remarquer la présence de l'espace entre le 'end' et le 'if' : end if

- La condition peut utiliser les variables définies ainsi que tous les opérateurs présents dans

SQL : =, >, <, <=, >=, <>, IS [NOT] NULL.

- Le '**else**' est facultatif , on peut trouver un 'if' avec ou sans 'else' :

```
If condition then  
Traitement ;  
End if ;
```

Exemple:

```
BEGIN  
    Update emp  
    Set sal=sal*1.1  
    Where empno = &numEmp;  
    If sql%found then  
        dbms_output.put_line('Un employé a été augmenté');  
    else  
        dbms_output.put_line('Pas d''employé augmenté');  
    end if;  
END;  
/
```

PL/SQL dispose d'une structure permettant d'exprimer la structure du choix multiple qui remplace en fait plusieurs structures 'if then else ' imbriquées, c'est la structure :

```
If condition1 then  
Traitements1 ;  
[elsif condition2 then /* elsif est sans e : */  
Traitements2 ;]  
[elsif condition3 then  
Traitements3 ;  
...  
[else  
Traitement ; ]  
end if /* un espace doit exister entre le end et le if ! */
```

Cette structure permet d'exprimer une similitude de choix et remplace l'utilisation des plusieurs 'if then else ' imbriquées.

- Le dernier 'else' est exécuté si aucune condition n'est valide.

```
Declare
    Vtype client.type %type;
    Vnom client.nom%type := 'Saddam';
    mess varchar2(15);
Begin
    Select nom,type into vnom,vtype from client where idclt=1000;
-- Contrôle de type client
    if vtype is null then
        mess:=vnom|| 'pas encore qualifié';
    elsif vtype='BON' then
        mess := vnom|| 'bon client ';
        update client set ca=ca*1.1 where idclt = 1000;
        Commit;
    else
-- type mauvais
        mess := vnom|| 'mauvais client ';
    end if;
    dbms_output.put_line('1000 '||mess);

end;
/
```

5.2 Traitements à choix multiples :

Syntaxe :

```
CASE expression
WHEN valeur THEN
instructions
[ , ... ]
[ ELSE
instructions ]
END CASE;
```

5.3 Les Traitements Répétitifs

Les traitements répétitifs permettent de répéter une suite de commandes, sans en répéter les instructions.

PL/SQL nous offre la possibilité d'effectuer des traitements répétitifs grâce à trois types d'instructions :

- Loop ..End loop
- for .. loop
- while .. loop.

a) Instruction Loop

Cette instruction représente la structure algorithmique 'itérer ... fin itérer', elle permet de répéter une séquence de commandes. Cette séquence est comprise entre le mot clé LOOP, indiquant le début d'une boucle et le END LOOP spécifiant sa fin. Cette instruction s'utilise évidemment dans le bloc begin ...end;

Sa syntaxe est :

```
LOOP
....
Instructions;
...
END LOOP
```

Pour pouvoir sortir de cette boucle, nous pouvons utiliser l'une des trois instructions suivantes :

- Exit : sortie inconditionnelle
- Exit when condition : sortie conditionnelle

```
If (condition) then exit ; end if;
```

Exemple :

```
Declare
Nombre number;
Begin
    Nombre:=0;
    loop
        Nombre := nombre+1;
        If nombre > 10 then
            Exit;
        End if;
    End loop;
End;
/
```

b) L'Instruction For ... Loop

La limitation des traitements répétitifs peut se faire en utilisant la clause FOR. Cette clause permet d'exécuter une séquence de commandes n fois. Le nombre d'itération est calculé par la différence entre les deux bornes.

La Syntaxe de cette instruction est :

```
For compteur in [reverse] borne_inférieur .. borne_supérieur
Loop
    Sequence_de_commandes;
End loop;
```

Compteur est une variable de type entier, locale déclarée dans la partie Declare. Sa valeur de départ est égale par défaut à borne_inférieure, elle s'incrémente de 1 après chaque traitement du contenu de la boucle, jusqu'à ce qu'elle atteigne la valeur limite qui est borne_supérieure. Le mot clé reverse permet d'utiliser une décrémentation de 1, en prenant comme valeur initiale borne_supérieure et comme valeur finale borne_inférieure.

Il est possible d'arrêter la boucle avant sa fin normale par une commande EXIT conditionnelle.

c) L'Instruction While .. Loop

La clause while permet d'exécuter le contenu d'une boucle tant qu'une condition spécifiée est vérifiée.

Sa syntaxe est :

```
While condition_PL/SQL
loop
...
instructions ;
...
End loop;
```

- La condition est une expression définie en combinant les opérateurs : <, >, =, !=, <=, >=, and, or.

```
-- calcul factoriel de 15
Declare
n number;
pf number :=1;
Begin
    n:=1;
    While (n <=15)
    loop
        pf := pf*n;
        n:=n+1;
    end loop;
    insert into res values ('factorielle 15 ',pf);
end;
/
```

6. Gestion des Exceptions (Erreurs)

Un programme PL/SQL s'exécutant peut provoquer des erreurs telles que l'accès à une donnée non existante dans la base, l'affectation de plusieurs données à une seule variable, une division par zéro ou une valeur erronée etc..

Quand une erreur système ou applicative se produit, une exception est générée.

Les traitements en cours dans la section d'exécution (section du BEGIN) sont stoppés. Les traitements de la section d'exception (section du EXCEPTION) commencent.

Ce mécanisme de gestion des erreurs permettra de définir le traitement approprié que le système doit exécuter en présence d'une erreur. Ce traitement se définit dans la partie lui est réservé dans le programme spécifiée par le mot EXCEPTION : en effet Le gestionnaire d'exception fait partie intégrante (mais optionnelle) du bloc PL/SQL et se trouve après les commandes. Il commence par le mot clé **EXCEPTION** et se termine avec le même END du bloc.

Un programme PL/SQL comportant une gestion d'erreurs aura la structure suivante :

```
Declare
....
Begin
....
Exception
Traitement des erreurs
End;
```

Les 4 types d'exception sont:

Type d'exception	Nommée	Anonyme (numérotée)
Système	1 : Exception système nommée	2 : Exception système anonyme
Utilisateur	3 : Exception utilisateur nommée	4 : Exception utilisateur anonyme

SQLCODE et SQLERRM

SQLCODE : pour afficher le numéro de l'erreur.

SQLERRM : pour afficher le numéro et le message d'erreur.

```
dbms_output.put_line('SQLCODE = '||SQLCODE);
dbms_output.put_line('SQLERRM = '||SQLERRM);
```

7.1 Les Erreurs internes(Système)

Les exceptions prédéfinies par ORACLE ne sont pas toutes nommées (il y en a trop). Par contre, elles ont toutes un numéro.

Les exceptions prédéfinies non nommées sont dites anonymes.

a) Exceptions prédéfinies nommées

- DUP_VAL_ON_INDEX : violation de l'unicité lors d'un insert ou d'un update .
- NO_DATA_FOUND : un select into ne ramène aucune ligne. A noter qu'avec une fonction du groupe, le select ramène toujours au moins une valeur, même si il n'y a pas de ligne (dans ce cas le retour vaut NULL).
- TOO_MANY_ROWS : un select into ramène plusieurs lignes.
- VALUE_ERROR : erreur de conversion de type.
- ZERO_DIVIDE : division par 0.
- Etc.

Pour gérer les exceptions, le développeur doit écrire un gestionnaire des exceptions qui reprend le contrôle du déroulement du bloc PL/SQL en présence d'une exception. Chaque gestion d'exception consiste à spécifier son nom d'erreur après la clause **WHEN** et la séquence de la commande à exécuter après le mot **THEN**, comme le montre l'exemple suivant :

```
Declare
vpu article.pu%type;
...
Begin
Select pu into vpu from article;
...
Exception
When Too_Many_Rows then .....;
-- traiter le cas où le select retourne plusieurs lignes
When No_Data_Found then .... ;
-- traiter le cas où le select ne retourne aucune ligne
when Others then ...;
..
End;
```

- La commande `select pu into vpu from article` va retourner :
- une seule ligne si la table contient un seul tuple ; dans ce cas aucune erreur ne sera générée par le noyau et le programme s'exécute correctement sans faire recours au gestionnaire des exceptions.
- Plusieurs lignes; le noyau génère dans ce cas l'erreur `Too_many_rows`, le système doit trouver une solution à ce problème, laquelle solution revient à définir le traitement que le système doit effectuer en présence de cette erreur due aux plusieurs lignes retournées par le `Select` et qui seront affectées à la variable `vpu` et qui n'est pas conforme à la syntaxe de la commande `select .. into .. from ...`; destinée à retourner une seule ligne.
- Aucune ligne retournée par le `Select` (table vide, ou table inexistante) ; de même le noyau génère l'erreur '**No_Data_Found**' et le programme pour qu'il puisse continuer à s'exécuter , on devait déjà prévu le traitement de ce cas en définissant le traitement de l'exception '**When No_Data_Found '**.
- Pour les autres erreurs éventuellement générées par le noyau et non nommées la clause - **Others** permet de spécifier le traitement convenable.

```
DECLARE
v_emp emp%rowtype;
v_empno emp.empno%type;
BEGIN
    v_empno:=&numEmp;
    SELECT * into v_emp from emp where empno=v_empno;
    dbms_output.put_line('Employé n° '||v_emp.empno);
```

```
        dbms_output.put_line('Nom : '||v_emp.ename||' - salaire : '||v_emp.sal);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        dbms_output.put_line('L''employé n° '||v_empno||' n''existe pas');
        dbms_output.put_line('SQLCODE = '||SQLCODE);
        dbms_output.put_line('SQLERRM = '||SQLERRM);
END;
/
Entrez une valeur pour numemp : 1000
ancien 5 : v_empno:=&numEmp;
nouveau 5 : v_empno:=1000;
L'employé n° 1000 n'existe pas
SQLCODE = 100
SQLERRM = ORA-01403: aucune donnée trouvée
```

b) exception système anonyme (WHEN OTHERS):

En déclarant une zone d'exception avec le cas « OTHERS », on prend en compte toutes les erreurs système.

```
DECLARE
    v_emp emp%rowtype;
    v_empno emp.empno%type;
BEGIN
    v_empno:=&numEmp;
    SELECT * into v_emp from emp where empno=v_empno;
    dbms_output.put_line('Employé n° '||v_emp.empno);
    dbms_output.put_line('Nom : '||v_emp.ename||' - salaire : '||v_emp.sal);
EXCEPTION
    WHEN OTHERS THEN
        dbms_output.put_line('SQLCODE = '||SQLCODE);
        dbms_output.put_line('SQLERRM = '||SQLERRM);
END;
/
Entrez une valeur pour numemp : 2000
ancien 5 : v_empno:=&numEmp;
nouveau 5 : v_empno:=2000;
SQLCODE = 100
SQLERRM = ORA-01403: aucune donnée trouvée
Procédure PL/SQL terminée avec succès.
```

- La fonction SQLCODE renvoie le code de l'erreur.
- La fonction SQLERRM renvoie le message de l'erreur.

Exercice :

Développer un programme PL/SQL qui calcule le chiffre d'affaire net d'un client(ca_net) dont le code est saisi. Ce programme affiche un message d'erreur si le code client est introuvable.

Ca_net = ca_brut*taux;

si nombre de commandes >100 alors

Taux = 1.1 *cn/nc

Sinon taux = 1.05*cn/nc

avec cn est le nombre des commandes client, nc est le nombre total des commandes.

```

Create table resultat(code varchar2(20), nom varchar2(20), chiffre_affaire number) ;
Accept idc Prompt 'Donner Code Client' :
Decalre
    Vnom client.nom%type;
    Ca commande.mntc%type;
    Nc integer ; -- nombre de commandes total
    Cn integer ; -- nombre de commandes client
    Taux real := 0.0;
Begin
--en cas d'echec ce select génère l'erreur data_not_found
    Select nom into vnom where idclt=&idc;
    Select sum (mntc) , count (nimc) into ca, cn from commande where idclt=&idc;
    Select count(numc) into nc from commande; -- si table commane est vide alors nc =0!
    If cn >100 then
        taux := 1.1*cn/nc;
    Else
        taux := 1.05*cn/nc;
    End if;
    Ca := ca*taux;
    Insert into resultat values (&idc,vnom, ca);
Exception
    When no_data_found then
        Insert into resultat (idc, 'Client not found', null);
        dbms_output.put_line('SQLCODE = '||SQLCODE);
        dbms_output.put_line('SQLERRM = '||SQLERRM);
    When zero_devide then -- table commande vide
        Insert into resultat (idc, 'Table commande vide !', null);
        dbms_output.put_line('SQLCODE = '||SQLCODE);
        dbms_output.put_line('SQLERRM = '||SQLERRM);
End;
/
select*from resultat ;

```

Le premier select peut échouer dans le cas où le code client est introuvable; l'exception interne `when no_data_found` s'exécute et empêche l'aboutement du programme. Si cette exception n'était pas traitée, l'exécution du programme ne sera pas correctement achevée et le système l'interrompt en affichant le code d'erreur interne.

7.2 Les Erreurs externes(Utilisateurs)

a) Exception utilisateur nommée

L'utilisateur peut gérer ses propres erreurs, c'est-à-dire des contraintes d'intégrité spécifiques non prises en compte par le SQL.

En déclarant une exception et en utilisant la fonction `RAISE nomException`, l'utilisateur renvoie le programme vers la zone `nomException` en cas d'erreur. L'exception est nommée..

La gestion des anomalies utilisateur peut se faire dans un bloc PL/SQL en effectuant les opérations suivantes :

```
Declare
- Nommer l'anomalie ( de type exception) dans la partie Declare du bloc.

-- déclaration de l'exception
Nom_ano exception;
Begin
....
- Déterminer l'erreur ( condition d'apparition) et passer la main au traitement approprié par
la commande Raise.
If (condition_anomalie ) then raise nom_anomalie;
Exception
- Effectuer le traitement défini dans la partie exception du bloc.
When (nom_ano) then (traitement );
.....
End;
```

Exemple:

```
Declare
    vpu article.pu%type;
    pu_zero exception ;
Begin
    Select pu into vpu from article;
    If vpu = 0 then
        Raise pu_zero;
    End if;
Exception
    When pu_zero then ....;
--- traitement exception externe pu_ezro
```

```

    When Too_Many_Rows then .....;
-- traiter le cas où le select retourne plusieurs lignes : erreur interne
    When No_Data_Found then .... ;
-- traiter le cas où le select ne retourne aucune ligne : erreur interne
    when Others then ...;
-- gérer toutes les autres exceptions
End;
```

Cet exemple montre comment on a défini une exception externe (non générée par le noyau) appelée `pu_zero` et qui se manifeste lorsque la commande `Select` retourne zéro dans la variable `vpu`. Son traitement se fait dans le paragraphe `Exception` du programme. Remarquez bien que l'exception externe contrairement à l'exception interne sa détection et son exécution sont explicitées par la condition **if condition then raise nom_exception;** alors qu'une exception interne sera détectée automatiquement par le noyau oracle et si on ne décrit pas le traitement approprié, il y' aura avortement du programme par le système en présence de telle erreur.

Exercice :

On considère qu'un nouveau salaire doit toujours être supérieur à l'ancienne valeur.

```

DECLARE
    UPDATE_SAL_EMP EXCEPTION;
    v_emp emp%rowtype;
    v_empno emp.empno%type;
    v_sal emp.sal%type;
BEGIN
    v_empno:=&numEmp;
    v_sal:=&salaire;
    SELECT * into v_emp from emp where empno=v_empno;
    dbms_output.put_line('Employé n° '||v_emp.empno);
    dbms_output.put_line('Nom : '||v_emp.ename||' - salaire : '||v_emp.sal);
    if v_sal < v_emp.sal then
        dbms_output.put_line('Le nouveau salaire : '||v_sal||
        ' ne peut pas être inférieur à l''ancien : '||v_emp.sal);
        RAISE UPDATE_SAL_EMP ;
    end if;
EXCEPTION
    WHEN UPDATE_SAL_EMP THEN
        dbms_output.put_line('Problème de mise à jour des données');
    WHEN NO_DATA_FOUND THEN
        dbms_output.put_line('L''employé n° '||v_empno||' n''existe pas');
        dbms_output.put_line('SQLCODE = '||SQLCODE);
        dbms_output.put_line('SQLERRM = '||SQLERRM);
END;
```

```
/
Entrez une valeur pour numemp : 7839
ancien 7 : v_empno:=&numEmp;
nouveau 7 : v_empno:=7839;
Entrez une valeur pour salaire : 4000
ancien 8 : v_sal:=&salaire;
nouveau 8 : v_sal:=4000;
Employé n° 7839
Nom : KING - salaire : 5000
Le nouveau salaire : 4000 ne peut pas être inférieur à l'ancien :
5000
Problème de mise à jour des données
Procédure PL/SQL terminée avec succès.
```

b) exception utilisateur anonyme

L'utilisateur peut gérer ses propres erreurs, c'est-à-dire des contraintes d'intégrité spécifiques non prises en compte par le SQL. En utilisant la fonction

RAISE_APPLICATION_ERROR, l'utilisateur renvoie le programme vers la zone OTHERS en cas d'erreur. L'exception n'est pas nommée.

Exemple :

On considère qu'un nouveau salaire doit toujours être supérieur à l'ancienne valeur.

```
DECLARE
    v_emp emp%rowtype;
    v_empno emp.empno%type;
    v_sal emp.sal%type;
BEGIN
    v_empno:=&numEmp;
    v_sal:=&salaire;
    SELECT * into v_emp from emp where empno=v_empno;
    dbms_output.put_line('Employé n° '||v_emp.empno);
    dbms_output.put_line('Nom : '||v_emp.ename||' - salaire : '||v_emp.sal);
    if v_sal < v_emp.sal then
        dbms_output.put_line('Le nouveau salaire : '||v_sal||' ne peut pas être inférieur
        à l''ancien : '||v_emp.sal);
        RAISE_APPLICATION_ERROR(-20000, 'Problème d''intégrité des
        données');
    end if;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        dbms_output.put_line('L''employé n° '||v_empno||' n''existe pas');
        dbms_output.put_line('SQLCODE = '||SQLCODE);
        dbms_output.put_line('SQLERRM = '||SQLERRM);
```

```
        WHEN OTHERS THEN
            dbms_output.put_line('SQLCODE = '||SQLCODE);
            dbms_output.put_line('SQLERRM = '||SQLERRM);
END;
/
Entrez une valeur pour numemp : 7839
ancien 6 : v_empno:=&numEmp;
nouveau 6 : v_empno:=7839;
Entrez une valeur pour salaire : 4000
ancien 7 : v_sal:=&salaire;
nouveau 7 : v_sal:=4000;
Employé n° 7839
Nom : KING - salaire : 5000
Le nouveau salaire : 4000 ne peut pas être inférieur à l'ancien :
5000
SQLCODE = -20000
SQLERRM = ORA-20000: Problème d'intégrité des données
Procédure PL/SQL terminée avec succès.
```

Précisions

La fonction **RAISE_APPLICATION_ERROR** a deux paramètres :

- Un numéro d'erreur, qu'on choisit entre -20 000 et -20 999 pour éviter les conflits avec les erreurs ORACLE.
- Un message d'erreur.

Ces deux paramètres sont ensuite utilisés par les fonctions SQLCODE et SQLERRM dans la zone d'exception **WHEN OTHERS**.

Annexe :

dept(deptno, dname, loc)

emp(empno, name, job, mgr, hiredate, sal, comm, #deptno)

Client(idclt, nom, ville, codp)

Article(idart, desart, PU, qtestk, qtemin)

Commande(numc, datec, mntc, idclt)

Lig_com(numc, idart, qtecom)